

TalentSync: Sistema para Empresa de Empleabilidad

Documento profesional de planificación del proyecto frontend

Equipo de desarrollo JobConnect:

- Erian Badilla Fallas
- Eiker Abarca Murillo
- Wayner Villalobos Arauz
- Palolo [nombre completo pendiente de confirmar]

Tecnología principal: Vite, JavaScript Vanilla, HTML y CSS

Versión: 1.0 - Planificación inicial (Borrador para aprobación interna) **Fecha**

de generación: 24 de agosto de 2026

Repositorio: [PENDIENTE DE DEFINIR POR EL EQUIPO]

Tabla 1: Control de versiones y estado del documento

Versión	Fecha	Responsable	Cambios realizados	Estado
1.0	24 de agosto de 2026	Equipo de desarrollo JobConnect	Planificación inicial del sistema, arquitectura, requisitos, pruebas y estrategia de trabajo.	Borrador para aprobación interna

Estado de aprobación: El contenido está listo para revisión del equipo. La asignación individual de

responsables y el enlace del repositorio deben completarse antes de iniciar el desarrollo.

3. Introducción

JobConnect es una propuesta de aplicación web para centralizar la administración de candidatos, vacantes, empresas clientes, postulaciones, entrevistas y tareas de reclutamiento. Su propósito es proporcionar al personal reclutador una vista operativa única, con navegación consistente, control de acceso y funciones de consulta, creación, actualización y eliminación de registros. El documento base establece que la solución debe ser únicamente frontend y consumir DummyJSON mediante Fetch API (JobConnect, s. f.).

La adopción de una API pública permite validar la integración HTTP y los flujos de interfaz sin construir un servidor propio. Fetch API ofrece una interfaz de JavaScript basada en promesas para ejecutar solicitudes HTTP y procesar sus respuestas; esta característica, combinada con `async/await`, permite expresar la secuencia de comunicación de forma legible y controlada (Mozilla Developer Network [MDN], s. f.-e).

La investigación apoyada por NotebookLM se integra como mecanismo de consulta y trazabilidad: las fuentes técnicas y del dominio se utilizan para justificar las decisiones, resolver dudas y mantener una bitácora. Este informe no sustituye la validación del equipo; organiza el alcance y separa los requisitos obligatorios, las decisiones propuestas, las recomendaciones y las limitaciones.

Tipología de decisiones del documento:

- **Requisito obligatorio:** Condición expresamente indicada en el enunciado y que debe implementarse.
- **Decisión técnica propuesta:** Alternativa planteada en esta planificación y sujeta a aprobación del equipo.
- **Recomendación:** Buena práctica que mejora calidad, seguridad o colaboración.
- **Limitación conocida:** Restricción técnica o de alcance que debe comunicarse y probarse.

4. Resumen del dominio de empleabilidad

4.1 Empleabilidad y servicios de empleo: La empleabilidad comprende el desarrollo y aprovechamiento de capacidades que facilitan el acceso, permanencia y movilidad de las personas en el trabajo. En Costa Rica, el Programa Nacional de Empleo vincula la formación, la intermediación laboral y la inserción de personas demandantes de empleo, al mismo tiempo que busca aportar talento a la productividad empresarial (Ministerio de Trabajo y Seguridad Social [MTSS], s. f.). En términos operativos, un servicio de empleo actúa como intermediario entre quienes buscan trabajo y las organizaciones que necesitan cubrir puestos; la Organización Internacional del Trabajo destaca que esa intermediación se fortalece cuando existe información organizada sobre competencias, demanda y oferta laboral (Organización Internacional del Trabajo [OIT], 2016).

4.2 Reclutamiento y selección: El reclutamiento reúne las actividades mediante las cuales una organización define una necesidad de talento, describe el puesto, comunica la vacante, recibe postulaciones y conforma un grupo de personas candidatas. La selección evalúa a quienes postulan para decidir quién se ajusta mejor al puesto. Una guía empresarial de referencia organiza el flujo general en definición del rol, publicación de la vacante, administración de solicitudes, entrevistas y oferta; esta secuencia se usa aquí únicamente como referencia del proceso, no como fuente jurídica aplicable a Costa Rica (Business.gov.uk, s. f.).

La selección suele incluir preselección y evaluación. Para que las decisiones sean comparables y trazables, conviene aplicar criterios claros y entrevistas estructuradas. El Chartered Institute of Personnel and Development señala que la entrevista permite valorar experiencia y capacidad para el puesto, mientras que la

estandarización de preguntas y criterios ayuda a disminuir sesgos (CIPD, 2024).

4.3 Entidades administradas

La plataforma digital de JobConnect conecta las siguientes entidades operativas para reducir la dispersión de información y aumentar la trazabilidad (OIT, 2025):

- **Reclutador:** Persona autenticada que consulta y actualiza la información de los procesos.
- **Candidato:** Persona cuyo perfil, datos de contacto y antecedentes relevantes se registran para procesos de selección.
- **Vacante:** Necesidad de contratación definida por una empresa, asociada con funciones, requisitos y condiciones.
- **Empresa cliente:** Organización que solicita apoyo para cubrir una o más vacantes.
- **Postulación:** Vínculo operativo entre una persona candidata y una vacante, acompañado de información para su evaluación.
- **Entrevista o nota:** Evidencia del contacto, la evaluación o el seguimiento de una postulación.
- **Tarea:** Acción pendiente asignada a un reclutador para mantener el avance y la trazabilidad del proceso.

5. Problema identificado

La gestión manual o fragmentada del reclutamiento puede distribuir datos entre hojas de cálculo, correos, mensajes y documentos sin una fuente operativa común. Esta situación dificulta localizar información actualizada, aumenta la duplicación de registros y reduce la capacidad de conocer el estado real de una postulación.

Consecuencias identificadas:

- Información dispersa de candidatos, vacantes y empresas clientes.
- Duplicación de datos y versiones contradictorias de un mismo registro.
- Seguimiento insuficiente de entrevistas, acuerdos y tareas pendientes.
- Dificultad para identificar responsables y estados de avance.
- Poca trazabilidad de creaciones, modificaciones y eliminaciones.
- Riesgo de omitir postulaciones o contactos por falta de recordatorios.
- Experiencia inconsistente para reclutadores cuando cada módulo utiliza criterios distintos.

6. Descripción del sistema

JobConnect será una aplicación web frontend desarrollada con Vite, JavaScript Vanilla, HTML y CSS. Un reclutador iniciará sesión, recibirá un token y accederá a un panel con seis módulos. Las vistas consumirán DummyJSON mediante fetch y ejecutarán los métodos permitidos por el documento base. La interfaz mostrará estados de carga, confirmaciones y mensajes de error, y se adaptará a distintos tamaños de pantalla.

La solución separará la autenticación, los servicios de API, la lógica de los módulos y las utilidades compartidas. Git y GitHub Flow respaldarán el trabajo colaborativo: cada cambio se desarrollará en una rama, se documentará mediante commits y se revisará en un pull request antes de integrarse a main (GitHub, s. f.-a).

7. Objetivo general

Desarrollar una plataforma web frontend para gestionar candidatos, vacantes, empresas clientes, postulaciones, entrevistas y tareas de reclutamiento mediante DummyJSON y Fetch API, incorporando autenticación con token, manejo asíncrono y colaboración controlada con Git.

8. Objetivos específicos

1. Implementar un inicio de sesión contra el endpoint /auth/login y almacenar el accessToken requerido por el proyecto.
2. Restringir las vistas administrativas cuando no exista una sesión válida y permitir el cierre de sesión seguro.
3. Consultar los registros de los seis módulos mediante solicitudes GET y presentar resultados comprensibles.
4. Crear registros mediante POST y actualizar información con PUT o PATCH según el método autorizado para cada módulo.
5. Eliminar registros mediante DELETE con confirmación previa y retroalimentación posterior.
6. Controlar errores de red, HTTP, autenticación, validación y procesamiento de respuestas sin interrumpir la aplicación.
7. Construir una interfaz responsiva y una organización modular del código que facilite mantenimiento y pruebas.
8. Coordinar el trabajo del equipo mediante ramas, commits descriptivos, revisiones y documentación en Git.

9. Alcance incluido

- Inicio y cierre de sesión con credenciales de prueba suministradas por DummyJSON.
- Protección de páginas o vistas administrativas mediante comprobación del token.
- Dashboard y navegación hacia seis módulos de negocio.
- Operaciones GET, POST, PUT, PATCH y DELETE únicamente donde correspondan.
- Formularios, validación básica, confirmación de eliminación y mensajes de éxito o error.
- Indicadores de carga y diseño responsivo.
- Arquitectura modular con servicios independientes para la API.
- Repositorio Git, estrategia de ramas, README y bitácora de consultas a NotebookLM.

10. Alcance excluido

- Backend propio, json-server o base de datos real.
- Persistencia permanente de creaciones, actualizaciones o eliminaciones en DummyJSON.

- Registro real de usuarios y recuperación de contraseñas.
- Envío de correos, notificaciones reales o carga permanente de archivos.
- Contrataciones, ofertas laborales o verificaciones legales reales.
- Despliegue productivo con controles de seguridad empresarial.
- Analítica avanzada, inteligencia artificial de selección o integración con bolsas de empleo reales.

11. Actores del sistema

Tabla 2: Actores y responsabilidades

Actor	Tipo	Responsabilidad o interacción	Acceso
Reclutador autenticado	Usuario principal	Inicia sesión y administra los seis módulos.	Interfaz protegida
Empresa cliente	Entidad administrada	Representa la organización que solicita talento.	Sin inicio de sesión
Candidato	Entidad administrada	Representa a la persona considerada para vacantes.	Sin inicio de sesión
DummyJSON	Servicio externo	Proporciona autenticación y recursos REST simulados.	Mediante Fetch API
Equipo de desarrollo	Actor técnico	Diseña, implementa, prueba y documenta la solución.	Repositorio y entorno local

Nota. En esta primera versión, el único usuario que inicia sesión es el reclutador. Las empresas y los candidatos son entidades administradas.

12. Tabla de módulos y endpoints

La URL base para todas las operaciones es <https://dummyjson.com>. La disponibilidad de métodos se mantiene exactamente como la define el documento base, aunque la documentación general de DummyJSON muestre otros métodos posibles (JobConnect, s. f.).

Tabla 3: Módulos, recursos y métodos HTTP autorizados

Módulo	Recurso	GET	POST	PUT	PATCH	DELETE	Propósito
Candidatos	/users	Sí	Sí	Sí	Sí	Sí	Administrar perfiles de candidatos (CRUD completo)
Vacantes	/products	Sí	Sí	Sí	Sí	Sí	Administrar oportunidades laborales (CRUD completo)
Empresas	/cars	Sí	Sí	Sí	No	Sí	Administrar empresas clientes (Sin PATCH)
Postulaciones	/posts	Sí	Sí	No	Sí	Sí	Relacionar candidatos y vacantes (Sin PUT)
Entrevistas	/comments	Sí	Sí	No	Sí	Sí	Registrar evaluación y seguimiento (Sin PUT)
Tareas	/todos	Sí	Sí	No	Sí	Sí	Controlar actividades pendientes (Sin PUT)

Nota. Requisito obligatorio tomado del enunciado. DummyJSON simula las mutaciones y no conserva permanentemente los cambios.

13. Mapeo de datos

Decisión técnica propuesta: Los siguientes nombres adaptan recursos genéricos de DummyJSON al

dominio de empleabilidad sin modificar la estructura real de sus respuestas. El equipo debe validar estas

etiquetas antes de implementar los formularios.

Tabla 4: Candidatos (recurso /users)

Campo DummyJSON	Nombre en JobConnect	Uso	Lista	Editable	Obligatorio
id	ID del candidato	Identificador de recurso	Sí	No	No (Solo lectura)
firstName	Nombre	Nombre de la persona	Sí	Sí	Sí
lastName	Apellidos	Apellidos de la persona	Sí	Sí	Sí
email	Correo	Contacto electrónico	Sí	Sí	Sí (Validar)
phone	Teléfono	Contacto telefónico	Sí	Sí	Sí
age	Edad	Dato demográfico	Sí	Sí	No
address.city	Ciudad	Ubicación de referencia	Sí	Sí	No (Anidado)
company.title	Perfil profesional	Cargo de referencia	Sí	Sí	No (Semántico)
university	Formación	Institución educativa	No	Sí	No (Detalle)

Tabla 5: Vacantes (recurso /products)

Campo DummyJSON	Nombre en JobConnect	Uso	Lista	Editable	Obligatorio
id	ID de vacante	Identificador de recurso	Sí	No	No (Solo lectura)
title	Nombre de la vacante	Título visible	Sí	Sí	Sí
description	Descripción	Funciones y contexto	Sí	Sí	Sí (Extracto)
category	Área profesional	Clasificación	Sí	Sí	Sí (Semántico)
price	Salario de referencia	Valor monetario simulado	Sí	Sí	No
stock	Cupos disponibles	Plazas simuladas	Sí	Sí	No (Semántico)
availabilityStatus	Estado	Disponibilidad	Sí	Sí	No
tags	Competencias	Palabras de clasificación	No	Sí	No

Tabla 6: Empresas clientes (recurso /carts)

Campo DummyJSON	Nombre en JobConnect	Uso	Lista	Editabl e	Obligatorio
id	ID de empresa	Identificador de recurso	Si	No	No (Semántico)
userId	ID de contacto	Referencia a usuario	Si	Si	Si
products	Vacantes asociadas	Arreglo de id/quantity	Si	Si	Si
totalProducts	Tipos de vacante	Cantidad de recursos	Si	No	No (API)
totalQuantity	Plazas requeridas	Suma de cantidades	Si	No	No (API)
total	Presupuesto bruto	Valor total simulado	Si	No	No
discountedTotal	Presupuesto ajustado	Valor con descuento	No	No	No

Tabla 7: Postulaciones (recurso /posts)

Campo DummyJSON	Nombre en JobConnect	Uso	Lista	Editabl e	Obligatorio
id	ID de postulación	Identificador de recurso	Si	No	No (Solo lectura)
title	Vacante postulada	Título de postulación	Si	Si	Si (Semántico)
body	Resumen de candidatura	Texto descriptivo	Si	Si	Si
tags	Estado y etiquetas	Clasificación	Si	Si	No (No nativo)
userId	ID del candidato	Vínculo con /users	Si	Si	Si (Debe existir)
reactions.likes	Valoraciones	Indicador cuantitativo	No	No	No (Calculado)
views	Revisiones	Conteo de consultas	No	No	No

Tabla 8: Entrevistas o notas (recurso /comments)

Campo DummyJSON	Nombre en JobConnect	Uso	Lista	Editabl e	Obligatorio
id	ID de nota	Identificador de recurso	Si	No	No (Solo lectura)
body	Nota de entrevista	Contenido del seguimiento	Si	Si	Si
postId	ID de postulación	Vínculo con /posts	Si	Si	Si
user.id	ID del reclutador	Referencia del autor	No	No	Si (Usa userId)
user.fullName	Reclutador	Nombre del autor	Si	No	No (API)
likes	Marcadores	Indicador simulado	No	No	No

Tabla 9: Tareas del reclutador (recurso /todos)

Campo DummyJSON	Nombre en JobConnect	Uso	Lista	Editabl e	Obligatorio
id	ID de tarea	Identificador de recurso	Si	No	No (Solo lectura)
todo	Descripción de tarea	Acción pendiente	Si	Si	Si
completed	Estado	Booleano pendiente/completada	Si	Si	Si
userId	ID del reclutador	Referencia al reclutador	Si	Si	Si

14. Requerimientos funcionales

Tabla 10: Matriz de requerimientos funcionales

Código	Descripción	Módulo	Prioridad	Forma de verificación
RF-01	Permitir iniciar sesión con usuario y contraseña contra /auth/login.	Autenticación	Alta	Login con datos válidos e inválidos.
RF-02	Almacenar el token y usarlo en peticiones mediante Authorization.	Autenticación/servicios	Alta	Inspección de localStorage y headers.
RF-03	Restringir el acceso si no existe un token válido.	Rutas protegidas	Alta	Intento de acceso sin token redirecciona .
RF-04	Cerrar sesión eliminando el token almacenado.	Autenticación	Alta	Logout y redirección al login.
RF-05	Listar mediante GET los registros de los seis módulos.	Seis módulos	Alta	Listado visible por módulo en interfaz.
RF-06	Crear mediante POST nuevos registros en cada módulo.	Seis módulos	Alta	Formulario y respuesta simulada de DummyJSON.
RF-07	Editar con PUT y/o PATCH según corresponda al módulo.	Módulos autorizados	Alta	Verificación de método y payload enviado.
RF-08	Eliminar mediante DELETE registros de cada módulo.	Seis módulos	Alta	Confirmación y respuesta isDeleted.
RF-09	Mostrar retroalimentación ante operaciones exitosas o fallidas.	Interfaz global	Alta	Mensajes de éxito y error visibles.
RF-10	Navegar entre los seis módulos desde una interfaz principal.	Dashboard	Alta	Prueba de navegación interactiva.

Nota. Los significados de RF-01 a RF-10 se mantienen conforme al enunciado. La prioridad es una propuesta del equipo.

15. Requerimientos no funcionales

Tabla 11: Matriz de requerimientos no funcionales

Código	Descripción	Categoría	Forma de verificación
RNF-01	Solución únicamente frontend con HTML, CSS y JavaScript.	Arquitectura	No existe backend propio ni base de datos real.
RNF-02	Código organizado en carpetas con estructura clara.	Organización	Revisión del árbol del proyecto en el entorno de desarrollo.
RNF-03	Consumo de datos con fetch y async/await.	Tecnología	Revisión de servicios JavaScript en la carpeta services.
RNF-04	Interfaz intuitiva, responsiva y usable.	Usabilidad	Pruebas en resoluciones de escritorio, tableta y móvil.
RNF-05	Errores controlados con try/catch sin romper la aplicación.	Confiabilidad	Simulación de desconexión y errores HTTP de la API.
RNF-06	Gestión con Git, repositorio remoto, ramas y commits frecuentes de todo el equipo.	Colaboración	Historial, ramas de Git y pull requests en GitHub.
RNF-07	Código legible, nombres significativos e indentación consistente.	Mantenibilidad	Revisión de código y formato consistente.
RNF-08	Credenciales y tokens sin exposición insegura en el código.	Seguridad	Búsqueda en repositorio y revisión de interfaz de usuario.
RNF-09	Inclusión de README.md con instrucciones de uso e instalación.	Documentación	README visible, probado y con comandos correspondientes.

Nota. Requerimientos obligatorios RNF-01 a RNF-09 derivados del documento base del proyecto (JobConnect, s. f.).

16. Flujo de autenticación

DummyJSON documenta un inicio de sesión mediante POST a `/auth/login` que devuelve `accessToken` y `refreshToken`. También muestra el uso de `Authorization: Bearer` para solicitar el usuario autenticado (DummyJSON, s. f.-a). Para JobConnect se aplicará el flujo obligatorio siguiente:

- El usuario abre la pantalla de inicio de sesión.
- Introduce usuario y contraseña.
- El frontend verifica que ambos campos tengan contenido.
- El servicio de autenticación ejecuta POST a `https://dummyjson.com/auth/login`.
- El cuerpo se envía como JSON con `Content-Type: application/json`.
- El servicio comprueba `response.ok` antes de procesar el resultado.
- La respuesta se procesa mediante `await response.json()`.
- El `accessToken` se almacena en `localStorage` con una clave única definida por el equipo.
- La interfaz redirige al dashboard.
- Cada vista protegida comprueba la existencia del token antes de mostrar el contenido.
- Las solicitudes que lo requieran agregan `Authorization: Bearer TOKEN`.
- Al cerrar sesión se elimina el token de `localStorage`.
- El usuario vuelve al login y no puede reingresar a vistas protegidas sin autenticarse.
- Cualquier fallo genera un mensaje comprensible y restablece el estado de carga.

16.1 Credenciales públicas de prueba:

- **Usuario:** emilys
- **Contraseña:** emilyspass

Aclaración: Son credenciales públicas proporcionadas por DummyJSON. Deben introducirse desde el formulario y no quedar escritas como valores fijos dentro de la lógica de autenticación.

17. Consideración de seguridad

El proyecto exige almacenar el token en `localStorage`. Esta decisión permite conservar el dato entre sesiones de navegación, característica documentada por MDN (s. f.-f). Sin embargo, OWASP advierte que los datos de `localStorage` son accesibles desde JavaScript y que una vulnerabilidad XSS podría exponer información sensible; para identificadores de sesión, recomienda mecanismos que puedan aprovechar cookies `HttpOnly` (OWASP Foundation, s. f.).

Medidas de mitigación en JobConnect:

1. Cumplir el requisito del documento base de usar `localStorage` únicamente para este prototipo.

2. No guardar la contraseña ni mostrar el token en la interfaz o la consola.
3. Eliminar el token al cerrar sesión.
4. Evitar insertar contenido no confiable mediante innerHTML; preferir textContent y validación de entradas.
5. No publicar tokens reales, archivos .env innecesarios ni capturas que revelen credenciales privadas.

Limitación conocida: La verificación de que exista un token en el navegador no equivale a una autorización

empresarial completa. Sin backend propio, la protección es demostrativa.

18. Funcionamiento de Fetch API y async/await

`fetch()` inicia una solicitud HTTP y devuelve una Promesa que se resuelve con un objeto `Response`. La Promesa representa un resultado que estará disponible posteriormente; no contiene de inmediato los datos finales. Una función declarada con `async` devuelve una Promesa, y `await` pausa la ejecución de esa función hasta que la operación esperada se resuelva o produzca un error (MDN, s. f.-a; MDN, s. f.-e).

El cuerpo de una respuesta se procesa de manera asíncrona. `response.json()` también devuelve una Promesa, por lo que debe esperarse antes de utilizar el objeto resultante. Además, `fetch` no rechaza automáticamente la Promesa por respuestas HTTP como 404 o 500: normalmente entrega un `Response` y corresponde al código revisar `response.ok` o `response.status`. `Response.ok` es verdadero para estados exitosos del rango 200-299 (MDN, s. f.-d; MDN, s. f.-e).

18.1 Patrón de servicio recomendado:

- Activar el estado de carga antes de iniciar la solicitud.
- Ejecutar `fetch` dentro de `try/catch`.
- Definir método, headers y body cuando correspondan.
- Comprobar `response.ok` y lanzar un error con contexto si es falso.
- Esperar `response.json()` y devolver los datos a la capa de interfaz.
- Registrar detalles técnicos solo en consola y mostrar un mensaje no técnico al usuario.
- Desactivar el estado de carga en `finally` o en una ruta equivalente garantizada.

19. Diferencia entre PUT y PATCH

PUT crea o reemplaza la representación completa de un recurso y se define como idempotente; PATCH aplica modificaciones parciales y no garantiza idempotencia por naturaleza (MDN, s. f.-b; MDN, s. f.-c; Dusseault & Snell, 2010).

Tabla 12: Comparación de PUT y PATCH en JobConnect

Criterio	PUT	PATCH
Propósito	Reemplazar la representación completa.	Modificar campos específicos.
Datos enviados	Objeto completo esperado por el servicio.	Solo propiedades que cambian.
Efecto	Puede sobrescribir campos omitidos según la API.	Conserva campos no incluidos.
Ejemplo en JobConnect	Sustituir todos los datos editables de una vacante.	Cambiar únicamente el estado de una tarea.
Riesgo	Perder datos si el payload está incompleto.	Aplicar una modificación sobre un estado desactualizado.
Módulos asignados	Candidatos, vacantes y empresas clientes.	Candidatos, vacantes, postulaciones, entrevistas y tareas.

Nota. La asignación de módulos responde al enunciado, incluso cuando DummyJSON documenta de forma general PUT o PATCH para un recurso (JobConnect, s. f.).

20. Manejo de errores

Tabla 13: Estrategia de manejo de errores

Escenario	Detección	Respuesta definida
Error de red	fetch rechaza la Promesa	Registrar detalle en consola; mostrar que no hay conexión; permitir reintento.
Error HTTP	response.ok es falso	Capturar status; lanzar error claro; mostrar mensaje contextual al usuario.
Credenciales incorrectas	Respuesta 400/401	No guardar token; informar en interfaz que los datos no son válidos.
Token inexistente	No existe clave en localStorage	Bloquear vista y redirigir inmediatamente al login.
Token vencido/inválido	Respuesta 401/403 en API	Eliminar token, informar al usuario y solicitar nuevo inicio de sesión.
JSON inválido	response.json() falla	Capturar SyntaxError y comunicar que la respuesta de API es inesperada.
Formulario incompleto	Validación en cliente	Marcar visualmente los campos vacíos requeridos antes de enviar la solicitud.
Eliminación fallida	DELETE sin respuesta exitosa	Conservar el registro visible en pantalla y permitir reintentar.
API no disponible	Timeout, 5xx o alta latencia	Finalizar visualización de carga; mensaje temporal; reintento controlado.

21. Arquitectura propuesta

La arquitectura separa la presentación, la autenticación y el acceso a datos. Vite se utilizará como herramienta de desarrollo y construcción; su documentación describe `index.html` como punto de entrada y ofrece una plantilla Vanilla para proyectos JavaScript (Vite, s. f.).

- **Presentación:** Páginas, componentes visuales, formularios, tablas, mensajes y navegación.
- **Módulos:** Lógica específica de candidatos, vacantes, empresas, postulaciones, entrevistas y tareas.
- **Servicios:** Funciones asíncronas que conocen endpoints, métodos, headers, body y manejo de respuestas.
- **Autenticación:** Inicio/cierre de sesión, acceso al token y protección de vistas (Auth Guard).

- **Utilidades:** Funciones compartidas de validación, carga, notificaciones y errores comunes.

22. Estructura de carpetas

Decisión técnica propuesta: La estructura utiliza Vite con JavaScript Vanilla y separa la lógica de API de la

interfaz. El equipo debe conservar los nombres aprobados para evitar importaciones inconsistentes.

```
jobconnect/  
  III index.html  
  III package.json  
  III README.md  
  III pages/  
    IIII dashboard.html  
    IIII candidatos.html  
    IIII vacantes.html  
    IIII empresas.html  
    IIII postulaciones.html  
    IIII entrevistas.html  
    IIII tareas.html  
  III src/  
    IIII assets/  
    IIII css/  
      IIIII global.css  
      IIIII auth.css  
      IIIII dashboard.css  
    IIII js/  
      IIIII auth/  
        IIIIII login.js  
        IIIIII authGuard.js  
      IIIII services/  
        IIIIII apiClient.js  
        IIIIII authService.js  
        IIIIII candidatosService.js  
        IIIIII vacantesService.js  
        IIIIII empresasService.js  
        IIIIII postulacionesService.js  
        IIIIII entrevistasService.js  
        IIIIII tareasService.js  
      IIIII modules/  
        IIIIII candidatos.js  
        IIIIII vacantes.js  
        IIIIII empresas.js  
        IIIIII postulaciones.js  
        IIIIII entrevistas.js  
        IIIIII tareas.js  
      IIIII utils/  
        IIIIII errors.js  
        IIIIII validators.js  
        IIIIII notifications.js  
        IIIIII loading.js  
  III docs/  
    IIII planificacion.docx  
    IIII bitacora-notebooklm.md
```


Tabla 14: Responsabilidad de las carpetas principales

Ruta	Responsabilidad
pages/	Vistas HTML protegidas y páginas de cada módulo.
src/assets/	Imágenes, iconos y recursos estáticos.
src/css/	Estilos globales, de la página de inicio de sesión y del panel administrativo.
src/js/auth/	Inicio de sesión, gestión de sesión, cierre de sesión y protección de vistas.
src/js/services/	Solicitudes a DummyJSON y procesamiento de respuestas HTTP.
src/js/modules/	Interacción de cada módulo con los formularios, listados y eventos de UI.
src/js/utils/	Funciones compartidas de validación, carga visual, avisos y errores.

23. Pantallas necesarias

Tabla 15: Pantallas, usuarios y acciones

Pantalla	Usuario	Función	Datos mostrados	Acciones	Requisito
Login	Reclutador	Autenticar	Formulario de usuario y contraseña	Ingresar	RF-01 a RF-04
Dashboard	Reclutador	Navegar y resumir	Accesos rápidos y estado	Abrir módulos; logout	RF-10
Candidatos	Reclutador	Gestionar perfiles	Campos de /users (Tabla 4)	Listar, crear, edit, del	RF-05 a RF-09
Vacantes	Reclutador	Gestionar vacantes	Campos de /products (Tabla 5)	Listar, crear, edit, del	RF-05 a RF-09
Empresas	Reclutador	Gestionar clientes	Campos de /carts (Tabla 6)	Listar, crear, edit, del	RF-05 a RF-09
Postulaciones	Reclutador	Gestionar candidaturas	Campos de /posts (Tabla 7)	Listar, crear, edit, del	RF-05 a RF-09
Entrevistas	Reclutador	Registrar notas	Campos de /comments (Tabla 8)	Listar, crear, edit, del	RF-05 a RF-09
Tareas	Reclutador	Controlar pendientes	Campos de /todos (Tabla 9)	Listar, crear, edit, del	RF-05 a RF-09

Nota. Cada módulo debe contemplar listado, formulario, edición, eliminación confirmada y estados de carga, éxito y error.

24. Estrategia de ramas

Se propone GitHub Flow, un flujo ligero basado en ramas. La documentación oficial recomienda trabajar en ramas descriptivas, realizar commits aislados, solicitar revisión mediante pull requests, resolver observaciones y fusionar (GitHub, s. f.-a).

Ramas propuestas para el equipo:

- **main:** Rama estable e integrable; no se trabaja directamente en ella.
- **feature/login y feature/dashboard:** Autenticación y navegación principal del sistema.
- **feature/candidatos, feature/vacantes, feature/empresas, feature/postulaciones, feature/entrevistas y feature/tareas:** Una rama por módulo de negocio.
- **feature/ui-responsive:** Ajustes visuales compartidos de CSS y adaptabilidad móvil.
- **docs/readme y docs/notebooklm:** Documentación, manuales y bitácora técnica.

24.1 Reglas de integración:

- Crear una rama desde main actualizada para cada funcionalidad independiente.
- Realizar commits pequeños, completos y descriptivos.

-

Subir cada rama al repositorio remoto y abrir un pull request (PR).

- Solicitar revisión de al menos otro integrante.
- Resolver conflictos y observaciones antes de fusionar.
- Eliminar la rama local y remota una vez integrada.

- Garantizar que todos los integrantes tengan contribuciones propias verificables en el repositorio.

25. Convención de commits

Se utilizarán prefijos semánticos para reconocer la intención de cada cambio en el historial de Git (GitHub, s. f.-a):

- **feat:** agregar formulario de inicio de sesión
- **feat:** implementar listado de candidatos
- **feat:** agregar creación de vacantes
- **fix:** corregir validación del token en rutas protegidas
- **fix:** manejar error al eliminar un registro
- **style:** mejorar diseño responsivo del dashboard
- **docs:** agregar planificación del proyecto
- **docs:** completar README
- **refactor:** centralizar llamadas a la API

26. Distribución del trabajo

Tabla 16: Matriz preliminar de asignación

Actividad	Responsable	Rama	Sesión	Evidencia	Estado
Investigación y planificación	[ASIGNAR]	docs/notebooklm	Día 1	Documento y fuentes	Pendiente
Login y protección	[ASIGNAR]	feature/login	Día 2	Commits, PR y prueba	Pendiente
Dashboard	[ASIGNAR]	feature/dashboard	Día 2	Navegación funcional	Pendiente
Candidatos (CRUD)	[ASIGNAR]	feature/candidatos	Día 2	CRUD y pruebas	Pendiente
Vacantes (CRUD)	[ASIGNAR]	feature/vacantes	Día 2	CRUD y pruebas	Pendiente
Empresas (CRUD)	[ASIGNAR]	feature/empresas	Día 2	CRUD y pruebas	Pendiente
Postulaciones (CRUD)	[ASIGNAR]	feature/postulaciones	Día 2	CRUD y pruebas	Pendiente
Entrevistas (CRUD)	[ASIGNAR]	feature/entrevistas	Día 2	CRUD y pruebas	Pendiente
Tareas (CRUD)	[ASIGNAR]	feature/tareas	Día 2	CRUD y pruebas	Pendiente
UI responsiva	[ASIGNAR]	feature/ui-responsive	Día 2-3	Pruebas visuales	Pendiente
README y cierre	[ASIGNAR]	docs/readme	Día 3	README y entregables	Pendiente

Nota. No se asignan responsables sin acuerdo del equipo. Erian Badilla Fallas, Eiker Abarca Murillo, Wayner Villalobos Arauz y Palolo deben participar activamente.

27. Plan de las tres sesiones

Tabla 17: Plan de ejecución por sesión

Sesión	Tareas de la sesión	Responsable	Evidencia	Estado
Día 1	Investigar fuentes; definir dominio, módulos, endpoints, requisitos, flujo de autenticación, estructura de carpetas y ramas; crear repositorio Git.	[Equipo]	Planificación y repositorio remoto	En curso
Día 2	Implementar inicio de sesión, protección de vistas, dashboard principal, seis módulos CRUD, control de errores y bitácora técnica mediante ramas.	[Equipo]	Aplicación funcional y Pull Requests	Pendiente
Día 3	Ejecutar pruebas del sistema, corregir incidencias, completar README.md, redactar documento reflexivo, video expositivo, infografía y entrega.	[Equipo]	Paquete de entrega completo	Pendiente

28. Plan de pruebas

Tabla 18: Casos mínimos de prueba

ID	Caso de Prueba	Acción ejecutada	Resultado esperado
P-01	Login correcto	Ingresar credenciales públicas válidas.	Token guardado en localStorage y redirección al dashboard.
P-02	Login incorrecto	Ingresar credenciales erróneas o vacías.	Mensaje claro de error en UI; token no se guarda.
P-03	Acceso sin token	Intentar abrir URL protegida (ej: candidatos).	Redirección inmediata al login.
P-04	Cierre de sesión	Seleccionar opción de logout en menú.	Token eliminado de localStorage y redirección a login.
P-05	GET seis módulos	Abrir el listado de cada uno de los módulos.	Datos visibles correctamente o estado vacío controlado.
P-06	POST seis módulos	Enviar formulario válido para creación.	Respuesta mostrada en pantalla y mensaje de éxito.
P-07	PUT autorizado	Editar candidatos, vacantes o empresas clientes.	Payload completo enviado y respuesta procesada.
P-08	PATCH autorizado	Editar campos específicos de postulaciones/ta reas.	Solo propiedades modificadas se envían a DummyJSON.
P-09	DELETE	Confirmar eliminación en modal o alerta.	Respuesta isDeleted de la API y retroalimentación en UI.
P-10	Cancelación DELETE	Cancelar la alerta de confirmación.	El registro permanece intacto en pantalla.
P-11	Error de red	Desconectar red o simular caída de la API.	Mensaje aclaratorio, estado de carga finaliza, app estable.
P-12	Error HTTP	Consultar un endpoint inexistente (404).	response.ok controlado y mensaje de error contextual.
P-13	Mensajes de UI	Forzar acciones exitosas y erróneas.	Retroalimentación clara y comprensible al usuario.
P-14	Responsividad UI	Probar la app en móvil, tableta y escritorio.	Sin desbordamiento de contenido ni controles inaccesibles.
P-15	Navegación	Recorrer las pantallas desde el menú.	Menú de navegación consistente y enlaces funcionales.

Nota. Cada prueba deberá registrar fecha, integrante que la ejecutó, resultado real y su respectiva evidencia.

29. Riesgos y mitigaciones

Tabla 19: Registro inicial de riesgos

Riesgo identificado	Prob.	Impacto	Medida preventiva / Plan de contingencia
DummyJSON no persiste cambios	Alta	Alto	Medida preventiva: Informar al usuario la limitación y diseñar estado local temporal. Contingencia: Recargar datos originales de la API y explicar comportamiento.
Caída o lentitud de la API	Media	Alto	Medida preventiva: Mostrar siempre estados de carga, timeouts y mensajes claros. Contingencia: Permitir reintentos controlados y documentar la incidencia.
Error CORS	Baja	Alto	Medida preventiva: Utilizar endpoints oficiales de DummyJSON y revisar cabeceras. Contingencia: Verificar URL, método y headers; registrar la duda en bitácora.
Token inválido en el navegador	Media	Alto	Medida preventiva: Controlar errores 401/403 de la API y validar la sesión local. Contingencia: Eliminar el token guardado y forzar redirección al login.
Exposición del token de acceso	Media	Alto	Medida preventiva: No imprimirlo en consola ni interfaz; reducir riesgos de XSS. Contingencia: Forzar cierre de sesión inmediato y corregir vulnerabilidad.
Conflictos en Git entre ramas	Media	Medio	Medida preventiva: Mantener ramas pequeñas, enfocadas e integradas frecuentemente. Contingencia: Resolver conflictos en equipo de forma sincrónica.
Pocos commits de un integrante	Media	Alto	Medida preventiva: Asignación clara de tareas y seguimiento diario en el equipo. Contingencia: Redistribuir responsabilidades de forma conjunta y documentar.
Diseño de UI inconsistente	Media	Medio	Medida preventiva: Definir una guía visual compartida y componentes comunes. Contingencia: Revisión visual final de la interfaz antes de la entrega.

Payloads incompatibles	Alta	Alto	Medida preventiva: Validar payloads enviados con la documentación de DummyJSON. Contingencia: Ajustar el mapeo semántico en el servicio sin alterar la API.
Falta de tiempo en sesiones	Media	Alto	Medida preventiva: Priorizar requisitos obligatorios y fijar metas diarias claras. Contingencia: Reducir mejoras estéticas opcionales, nunca requisitos.

30. Bitácora de NotebookLM

Tabla 20: Plantilla de registro de consultas

Fecha	Integrante	Fase	Problema / Pregunta formulada	Fuente y Solución aplicada / Resultado
--	[Equipo]	Planificación	Plantilla para registrar las consultas técnicas realizadas a NotebookLM durante el proyecto. Se deben documentar preguntas, fuentes y soluciones aplicadas.	Consumo de DummyJSON / Pendiente de completar por el equipo.

30.1 Ejemplos de preguntas útiles:

- *"Según MDN y DummyJSON, ¿qué validaciones debo realizar después de ejecutar fetch antes de procesar response.json()?"*
- *"¿Qué diferencia práctica existe entre PUT y PATCH y cuáles módulos de JobConnect admiten cada método según el enunciado?"*
- *"¿Cómo debe viajar un Bearer Token en el encabezado Authorization y qué respuesta debe controlar la interfaz si deja de ser válido?"*
- *"¿Qué campos reales del recurso /carts pueden adaptarse a empresas clientes sin alterar la estructura de DummyJSON?"*
- *"¿Qué pasos de una entrevista estructurada requieren trazabilidad y cómo pueden representarse mediante /comments en este prototipo?"*

31. Limitaciones conocidas

- DummyJSON no conserva permanentemente las operaciones POST, PUT, PATCH y DELETE (las mutaciones son simuladas).
- El sistema es únicamente frontend y no posee una base de datos propia para persistencia empresarial.
- La autenticación y la protección de vistas mediante Auth Guard tienen fines estrictamente demostrativos en cliente.
- El token se almacena en localStorage por requisito del enunciado, conociendo el riesgo de seguridad frente a XSS.

- Los recursos de DummyJSON se adaptan semánticamente a empleabilidad y no constituyen modelos empresariales reales.
- Los cambios simulados pueden desaparecer por completo al recargar la página o volver a consultar la API.
- La plataforma no realiza contrataciones, ofertas, verificaciones legales ni decisiones de selección reales.
- Los campos de candidatos provienen de datos de prueba (/users) y no representan expedientes reales.

32. Entregables del proyecto

Tabla 21: Control de entregables

Entregable	Estado	Responsable	Enlace o evidencia de entrega	Observaciones
Documento de planificación	Listo para revisión	Equipo de desarrollo	Este archivo PDF	Completar repositorio y nombres definitivos antes de la entrega final.
Repositorio Git	Pendiente	[Equipo]	[Por definir]	No afirmar la creación sin un enlace remoto funcional.
Aplicación frontend	Pendiente	[Equipo]	[Por definir]	Día 2-3. Incluye login, dashboard y seis módulos.
Inicio de sesión (Login)	Pendiente	[Equipo]	[Por definir]	Autenticación, token en localStorage y Auth Guard.
Seis módulos CRUD	Pendiente	[Equipo]	[Por definir]	Implementación de los seis CRUD utilizando Fetch API.
Bitácora de NotebookLM	Preparada	[Equipo]	Sección 30 del documento	Registrar las consultas técnicas reales efectuadas.
Video de presentación	Pendiente	[Equipo]	[Por definir]	Exposición del sistema y flujo técnico (Día 3).
Infografía del dominio	Pendiente	[Equipo]	[Por definir]	Diseño gráfico sobre el dominio de empleabilidad (Día 3).
README.md	Pendiente	[Equipo]	[Por definir]	Instrucciones de instalación y ejecución del proyecto local.
Documento reflexivo	Pendiente	[Equipo]	[Por definir]	Ensayo de la experiencia de codesarrollo con IA.

33. Conclusión

La planificación define el problema, alcance, actores, requisitos, arquitectura, mapeo de datos, autenticación, manejo de errores, estrategia colaborativa y pruebas de JobConnect. Las fuentes técnicas y de empleabilidad sustentan estas decisiones y reducen inconsistencias durante la implementación. Antes de iniciar el Día 2, el equipo debe aprobar el mapeo de campos, asignar responsables y crear el repositorio; después priorizará autenticación, navegación y servicios compartidos antes de completar los seis módulos.

34. Referencias bibliográficas (Normas APA 7)

- Business.gov.uk. (s. f.). *Recruiting people for your business*. Recuperado el 24 de agosto de 2026, de <https://www.business.gov.uk/support/employment-skills/recruiting-people-for-your-business/> Chartered Institute of Personnel and Development. (2024, 11 de noviembre). *Selection methods*.
<https://www.cipd.org/uk/knowledge/factsheets/selection-factsheet/>
- DummyJSON. (s. f.-a). *Auth*. Recuperado el 24 de agosto de 2026, de <https://dummyjson.com/docs/auth>
- DummyJSON. (s. f.-b). *Carts*. Recuperado el 24 de agosto de 2026, de <https://dummyjson.com/docs/carts>
- DummyJSON. (s. f.-c). *Comments*. Recuperado el 24 de agosto de 2026, de
<https://dummyjson.com/docs/comments>
- DummyJSON. (s. f.-d). *Posts*. Recuperado el 24 de agosto de 2026, de <https://dummyjson.com/docs/posts>
- DummyJSON. (s. f.-e). *Products*. Recuperado el 24 de agosto de 2026, de
<https://dummyjson.com/docs/products>
- DummyJSON. (s. f.-f). *Todos*. Recuperado el 24 de agosto de 2026, de <https://dummyjson.com/docs/todos>
- DummyJSON. (s. f.-g). *Users*. Recuperado el 24 de agosto de 2026, de <https://dummyjson.com/docs/users> Dusseault, L., & Snell, J. (2010). *PATCH method for HTTP (RFC 5789)*. RFC Editor.
<https://www.rfc-editor.org/rfc/rfc5789>
- GitHub. (s. f.-a). *GitHub flow*. Recuperado el 24 de agosto de 2026, de
<https://docs.github.com/en/get-started/using-github/github-flow>
- GitHub. (s. f.-b). *Quickstart for repositories*. Recuperado el 24 de agosto de 2026, de
<https://docs.github.com/en/repositories/creating-and-managing-repositories/quickstart-for-repositories>
- JobConnect. (s. f.). *Sistema para Empresa de Empleabilidad: Plataforma web JobConnect - Consumo de servicios con Fetch API* [Documento de trabajo no publicado].
- Jones, M., & Hardt, D. (2012). *The OAuth 2.0 authorization framework: Bearer token usage (RFC 6750)*. RFC Editor.
<https://www.rfc-editor.org/rfc/rfc6750>
- Ministerio de Trabajo y Seguridad Social. (s. f.). *Programa Nacional de Empleo - PRONAE*. Recuperado el 24 de agosto de 2026, de <https://www.mtss.go.cr/empleo-formacion/pronae.html>
- Mozilla Developer Network. (s. f.-a). *async function*. Recuperado el 24 de agosto de 2026, de
https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Statements/async_function
- Mozilla Developer Network. (s. f.-b). *PATCH*. Recuperado el 24 de agosto de 2026, de
<https://developer.mozilla.org/es/docs/Web/HTTP/Reference/Methods/PATCH>
- Mozilla Developer Network. (s. f.-c). *PUT*. Recuperado el 24 de agosto de 2026, de
<https://developer.mozilla.org/es/docs/Web/HTTP/Reference/Methods/PUT>
- Mozilla Developer Network. (s. f.-d). *Response.ok*. Recuperado el 24 de agosto de 2026, de
<https://developer.mozilla.org/es/docs/Web/API/Response/ok>

Mozilla Developer Network. (s. f.-e). *Using the Fetch API*. Recuperado el 24 de agosto de 2026, de https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

Mozilla Developer Network. (s. f.-f). *Window.localStorage*. Recuperado el 24 de agosto de 2026, de <https://developer.mozilla.org/es/docs/Web/API/Window/localStorage>

Organización Internacional del Trabajo. (2016, 26 de enero). *The role of employment service providers: Guide to anticipating and matching skills and jobs* (Vol. 4).
<https://www.ilo.org/publications/role-employment-service-providers-guide-anticipating-and-matching-skills>

Organización Internacional del Trabajo. (2025, 30 de julio). *How to support an employer? Manual for managerial and operational teams in PES*.
<https://www.ilo.org/publications/how-support-employer-manual-managerial-and-operational-teams-pes>

OWASP Foundation. (s. f.). *HTML5 security cheat sheet*. Recuperado el 24 de agosto de 2026, de https://cheatsheetseries.owasp.org/cheatsheets/HTML5_Security_Cheat_Sheet.html

Vite. (s. f.). *Getting started*. Recuperado el 24 de agosto de 2026, de <https://vite.dev/guide/>

35. Lista final de verificación (Checklist de cumplimiento)

- **Cumple:** Descripción del sistema completa.
- **Cumple:** Tabla de módulos y endpoints completa, con métodos exactos.
- **Cumple:** Requerimientos funcionales RF-01 a RF-10 y RNF-01 a RNF-09 incluidos.
- **Cumple:** Flujo de autenticación definido rigurosamente.
- **Cumple:** Resumen del dominio de empleabilidad formalmente redactado.
- **Cumple:** Estructura de carpetas detallada y modular.
- **Cumple:** Estrategia de ramas, commits y reglas de integración de Git.
- **Cumple:** Matriz preliminar de distribución de trabajo.
- **Cumple:** Planificación del cronograma por sesiones.
- **Cumple:** Plan de pruebas exhaustivo con 15 casos clave.
- **Cumple:** Matriz de riesgos y planes de mitigación/contingencia.

- **Cumple:** Bitácora preparada de NotebookLM.
- **PENDIENTE:** Crear el repositorio e incorporar su enlace final.
- **PENDIENTE:** Asignar responsables definitivos a cada actividad.
- **PENDIENTE:** Confirmar el nombre completo de Palolo para la versión profesional final.